

Plan de Pruebas y Resultados

Campana de Aseguramiento de Calidad (QE)

Microservice Code Studio — AgentIA

Equipo de Ingeniería AgentIA

29 de septiembre de 2026

Para revisión del Project Manager. Este documento describe qué se probó, cómo, qué se encontró y qué falta. Las cifras de la Sección 7 se obtuvieron ejecutando la suite completa dos veces sobre el mismo entorno: una sobre el código anterior (punto de partida) y otra sobre el código actual. El comando exacto de reproducción está en la Sección 7. Todos los registros crudos quedan versionados en `reports/qe/`.

Contents

1	Resumen ejecutivo	2
2	Objetivo y alcance	2
2.1	Objetivo	2
2.2	Alcance acordado	2
2.3	Fuera de alcance	3
3	Estrategia de pruebas	3
3.1	Enfoque basado en riesgo	3
3.2	Sustitución de fronteras	3
3.3	Hermeticidad	4
4	Inventario de pruebas automatizadas	4
5	Matriz de trazabilidad	4
6	Defectos encontrados y corregidos	5
7	Resultados de ejecución	8
8	Brechas restantes y recomendaciones	9
9	Limitaciones del informe	10

1 Resumen ejecutivo

La campaña tuvo dos fases. La primera cubrió los tres módulos con mayor riesgo de producto según el inventario de la campaña anterior (`docs/qe_test_matrix.md`); la segunda cerró el motor del Auto-Pilot y añadió pruebas de flujo del frontend en la frontera HTTP. El criterio no fue “el número más bajo”, sino *qué decisión toma cada módulo y cuánto duele si la toma mal*.

Módulo	Antes	Después
<code>services/docker_service.py</code>	52,7 %	100,0 %
<code>api/routes_session.py</code>	71,3 %	100,0 %
<code>services/lifecycle_service.py</code>	76,4 %	100,0 %
<code>services/pipeline_runner.py</code>	82,4 %	100,0 %
Sentencias sin cubrir	994	727
Pruebas de backend	589 correctas	825 correctas
Pruebas de frontend	64 (10 archivos)	73 (11 archivos)

La campaña añadió **186 pruebas automatizadas** (177 de backend, 9 de frontend) y encontró **cinco defectos reales**. Ninguno era visible leyendo el código: todos aparecieron porque una prueba falló, se colgó, o ejecutó por primera vez una rama que nunca se había ejecutado.

- **D-03 (crítico)**. El espejo de telemetría de costos, documentado como “best-effort, fuera de toda ruta crítica”, *bloqueaba* la ruta crítica. Un destino inalcanzable detenía cada llamada al modelo durante minutos, y detenía la suite completa de forma indefinida. Con el defecto presente, la cobertura del backend no podía medirse en absoluto.
- **D-05 (alto)**. El respaldo de síntesis de esquema SQL — añadido precisamente para no entregar un esquema equivocado — llamaba a un método que no existe, dejaba un `schema.sql` *vacío* y tumbaba la ejecución completa.
- **D-02 (alto)**. Las fases 5 y 6 mostraban el estado `BLOCKED` con `blockingReason: null`: el operador veía que el pipeline se había detenido, pero no por qué. Afecta directamente al Principio V de la constitución.
- **D-04 (alto)**. La decisión de modo y la carga del conjunto de instrucciones ocurrían *fuera* del manejador de errores del pipeline, de modo que un fallo ahí escapaba del hilo demonio y dejaba la sesión en `RUNNING` para siempre.
- **D-01 (medio)**. Dos funciones del ciclo de vida estaban definidas dos veces en el mismo archivo. La primera definición era código muerto e imposible de cubrir, y su contrato *difería* del de la segunda.

Los cinco están corregidos y fijados con pruebas de regresión. La Sección 6 documenta causa raíz y evidencia de cada uno.

2 Objetivo y alcance

2.1 Objetivo

Incorporar pruebas automatizadas ejecutables y deterministas sobre las superficies del producto con mayor cobertura pendiente, y entregar evidencia verificable de su resultado para la revisión del Project Manager.

2.2 Alcance acordado

La campaña cubre **tres módulos de backend**, elegidos por riesgo:

1. `services/docker_service.py` — la ruta de despliegue local que la interfaz de demostración opera directamente (40,4 % de cobertura según la campaña anterior).
2. `api/routes_session.py` — la superficie principal del producto: creación, listado, lectura, cancelación de sesiones, el flujo SSE del monitor y los caminos de desenlace de la generación (69,9 %).
3. `services/lifecycle_service.py` — la máquina de estados de fases: qué se puede entrar, qué porcentaje está completo y qué se recomienda a continuación (76,4 %).

2.3 Fuera de alcance

- Pruebas de aceptación independientes contra un microservicio generado y desplegado (la limitación de la Sección 9); requieren Docker y una compilación real.
- Pruebas de navegador de extremo a extremo (Playwright). El frontend *sí* recibió pruebas de flujo, pero en jsdom y contra un backend simulado, no en un navegador real contra servidores vivos.
- `ssl_compat.py` y otras rutas de valor bajo.

3 Estrategia de pruebas

3.1 Enfoque basado en riesgo

Cada módulo se abordó preguntando primero *qué afirmación hace y qué pasa si la hace mal*, y sólo después qué líneas faltaban. Las afirmaciones que se fijaron son:

- `docker_service`: `HEALTHY` es una **afirmación de verificación**. Sólo es honesta si un `/actuator/health` real respondió `UP`. Que `docker compose up` salga con código 0 significa “los contenedores arrancaron”, nunca “el servicio funciona”.
- `routes_session`: los caminos de desenlace deben decir la verdad. Una sesión que llegó a `VERIFIED` sobre una verificación *sustituída* debe declararlo (`verificationFallbackUsed`, feature 012); una sesión bloqueada debe registrarse igual que una completada (feature 015); un fallo dentro del grafo debe terminar la sesión, no dejarla en `RUNNING` para siempre.
- `lifecycle_service`: los prerrequisitos de fase son una regla de negocio. El límite de 3 auto-reparaciones (Principio V) debe cerrar la puerta, no sólo pintarse.

3.2 Sustitución de fronteras

Ninguna prueba nueva inicia un contenedor, abre un socket ni compila Java. Las fronteras externas se sustituyen siempre en el mismo punto:

Frontera	Sustituto	Motivo
<code>subprocess.Popen/run</code>	proceso falso con salida guionada	sin Docker
<code>requests.get</code>	respuesta falsa o excepción	sin red
<code>threading.Thread</code>	hilo en línea	determinismo
<code>generation_graph</code>	grafo guionado	sin LLM
<code>audit_workspace</code>	auditoría falsa	sin SAST

El hilo del `docker compose` merece mención aparte. `deploy_local` devuelve el control en cuanto lanza el hilo trabajador; una prueba que esperase a que terminara sería lenta en una máquina descargada y fallida en una cargada. Sustituyendo `Thread` por una implementación que ejecuta el objetivo en línea, la prueba observa un despliegue *terminado* sin dormir ni sondear.

3.3 Hermeticidad

La suite quedó aislada de la red. Las tres tiendas de estado a nivel de módulo (`_active_deployments`, `SESSION_EVENT_HISTORY`, `SESSION_EVENT_SUBSCRIBERS`) se aíslan por prueba, porque son globales al proceso por diseño y sin aislamiento una prueba devuelve la sesión de otra. El destino de telemetría se redirige a un almacén local temporal (Sección 6).

4 Inventario de pruebas automatizadas

Archivo	Pruebas	Superficie que cubre
<code>test_qe_deploy_docker.py</code>	33	Detección del demonio Docker; <code>deploy_local</code> (éxito, sin demonio, <code>compose</code> con error, smoke test fallido, excepción del trabajador, <code>rebuild</code>); <code>run_smoke_test</code> (UP, estado degradado, cuerpo no JSON, reintentos, éxito tardío); <code>get_deployment_status</code> ; <code>stream_logs</code> ; <code>stop_deployment</code> .
<code>test_qe_session_surface.py</code>	50	Lectura y persistencia de <code>verificationFallbackUsed</code> ; difusión de eventos SSE y suscriptores caídos; <code>stream_session_events</code> (replay, evento en vivo, keep-alive, desconexión); <code>list_sessions</code> (promoción a COMPLETED, fallo tolerado); <code>quick_start_session</code> ; cancelación; y los cuatro desenlaces de <code>execute_generation_pipeline</code> .
<code>test_qe_lifecycle_rules.py</code>	46	Detección de las 7 fases; porcentaje de avance; prerequisites; límite de 3 auto-reparaciones; Quality Gate bloqueado y auditoría que falla; fases obsoletas (marcado, acumulación, limpieza, JSON corrupto); degradación de valores de enumeración desconocidos; estado del pipeline; <code>get_project_overview</code> .
<code>test_cost_mlflow_mirror.py</code>	+2	Cota de transporte del espejo de telemetría (sin reintentos, tiempo de espera acotado) y respeto de una configuración explícita del operador.
<code>test_qe_pipeline_autopilot.py</code>	46	Pausa cooperativa en cada frontera de paso; cancelar frente a pausar; los puntos de entrada <code>pause/resume/cancel</code> y la guarda de concurrencia de <code>run_pipeline</code> ; <code>stream_pipeline_events</code> completo (conexión, progreso, heartbeats, cierre); el camino de agotamiento; el respaldo de esquema y el de arquitectura; y la propagación de errores.
<code>workflow_journeys.test.tsx</code> (frontend)	9	Flujos de usuario contra un backend simulado en la frontera HTTP con MSW: inicio rápido y lectura de vuelta, ciclo de control del Auto-Pilot, resumen y ciclo de vida, artefactos, credencial efímera, propagación de fallos, y el flujo en la interfaz incluido el informe honesto del smoke test.

Total: **186 pruebas nuevas** (177 de backend, 9 de frontend). Las campañas anteriores (`test_qe_api_surface.py`, `test_qe_environment_repair.py`, `test_qe_publish_git.py`, `test_qe_schema_consistency.py`) permanecen y siguen pasando: 67 pruebas más.

5 Matriz de trazabilidad

Requisito / Principio	Afirmación verificada y prueba que la fija
Principio V — auto-reparación acotada a 3 intentos	Con 3 intentos la fase 5 queda BLOCKED y <code>canAdvance</code> es falso; con 2 no. La razón del bloqueo llega al cliente. <code>test_a_blocked_repair_loop_blocks_phase_five...</code> , <code>test_two_failed_repairs_do_not_block_phase_five</code> , <code>test_a_repair_beyond_the_limit_is_reported_as_blocked</code>
Feature 012 (FR-005) — la verificación sustituida debe declararse	El indicador se lee del registro persistido (sobrevive a un reinicio), se escribe en el evento terminal y es una medición, no una constante. <code>test_reading_a_session_surfaces_the_persisted_fallback_flag</code> , <code>test_a_completed_run_records_verified_and_reports_the_substituted_verification</code> , <code>test_a_completed_run_with_a_real_verification_reports_no_fallback</code>
Feature 015 (FR-001) — el diagnóstico se registra en todo camino terminal	Tanto el camino COMPLETED como el BLOCKED invocan al escritor único. <code>test_a_completed_run_records_verified_and_reports_the_substituted_verification</code> , <code>test_a_blocked_run_is_persisted_and_diagnosed</code>
Honestidad de la verificación (feature 012)	<code>compose</code> con código 0 y health degradado produce FAILED, nunca HEALTHY. <code>test_a_container_that_never_becomes_healthy_is_failed_not_healthy</code> , <code>test_a_missing_health_field_is_not_treated_as_up</code>
Principio VI — cero secretos, cero red en pruebas	La suite no realiza ninguna conexión saliente; el token de Git no sobrevive a la publicación. <code>no_outbound_telemetry</code> (conftest), <code>test_qe_publish_git.py</code>
Principio III — sobre de error uniforme	Los manejadores devuelven <code>status/message/details</code> . <code>test_qe_api_surface.py</code> (campana anterior)
Honestidad de la verificación, de extremo a extremo	Cuando el endpoint de salud falla, la interfaz muestra “Smoke test NO ejecutado”; nunca un éxito que nadie observó. Un fallo del backend se propaga en lugar de inventarse. <code>workflow_journeys.test.tsx</code>

6 Defectos encontrados y corregidos

Los tres defectos siguientes fueron encontrados *por las pruebas*, no por lectura del código. Se documentan con severidad, evidencia, causa raíz y prueba de regresión.

D-03 — El espejo de telemetría “best-effort” bloqueaba la ruta crítica

Severidad: crítica. Afecta a producción y al arnés de pruebas.

Evidencia. La suite completa no terminaba. Se detenía indefinidamente en `test_cost_recording.py::test_a_call_through_the_stage_boundary_is_recorded_end_to_end`. La traza del proceso colgado, obtenida con `faulthandler`, muestra la pila exacta:

```
urllib3/util/retry.py      _sleep_backoff
requests/adapters.py      send
mlflow/tracking/fluent.py start_run
app/cost/mlflow_sink.py:49 _record
app/cost/recording.py:148 _record_call
app/orchestrator/stages/runner.py:994 _run_model_stage
backend/tests/test_cost_recording.py:116
```

Causa raíz. `mlflow_sink.py` se declara “deliberadamente fuera de toda ruta crítica” y sostiene que un destino inalcanzable “simplemente no hace nada”. El transporte no se comportaba así. Los valores

por defecto de MLflow son `MLFLOW_HTTP_REQUEST_TIMEOUT = 120` segundos y `MLFLOW_HTTP_REQUEST_MAX_RETRIES = 7` con retroceso exponencial (factor 2); la propia documentación de la librería calcula unos 4 minutos para esas 7 tentativas. El destino por defecto es `http://localhost:5000`, que en este entorno no existe.

Y el llamador *sí* está en la ruta crítica: `mirror_call_record` se invoca desde `RecordingChatClient.invoke`, que envuelve **cada** llamada al modelo. Por tanto, una caída de telemetría detenía cada sesión de generación durante minutos. El módulo no era ni “fuera de ruta” ni silencioso.

Corrección. Se añadió `_bound_the_transport()`, que fija las dos únicas variables que MLflow consulta en cada petición (`MLFLOW_HTTP_REQUEST_TIMEOUT = 5`, `MLFLOW_HTTP_REQUEST_MAX_RETRIES = 0`) mediante `setdefault`. Se usa `setdefault` y no asignación directa para que un despliegue con un servidor MLflow sano pueda pedir reintentos largos sin que el espejo se los quite.

Corrección del arnés. El destino se redirige a un almacén local temporal en `conftest.py`. Se redirige en lugar de anularse porque `test_the_local_store_is_written_even_when_the_mirror_is_absent` afirma que el espejo *sí* fue consultado; anularlo habría vuelto vacía esa prueba. La dirección se parchea sobre la configuración y no sobre el ayudante `_tracking_uri`, para que el ayudante siga bajo prueba.

Resultado. La prueba que colgaba indefinidamente ahora tarda 1,45 s. La suite completa termina. Se añadieron dos pruebas de regresión que fijan la cota de transporte y el respeto de la configuración explícita.

D-05 — El respaldo del esquema llamaba a un método inexistente

Severidad: alta. Rompía la ejecución completa y dejaba un artefacto vacío.

Evidencia. La primera prueba que ejecutó la rama de respaldo del esquema falló con `AttributeError`. El archivo `schema.sql` existía, pero estaba *vacío*. La ejecución terminaba en `FAILED`.

Causa raíz. `schema_sql_from_draft` es una función de módulo, no un método del objeto `model_sql_service`. El respaldo — introducido por el commit 37b5853 justamente para no entregar un esquema equivocado — la invocaba como `model_sql_service.schema_sql_from_draft(draft)`, de modo que lanzaba `AttributeError`. Y como `open(..., "w")` trunca el archivo *antes* de evaluar su argumento, el resultado era un `schema.sql` vacío además de la caída.

Nadie lo detectó porque la rama nunca se ejecutó: estaba en el conjunto de líneas sin cubrir. La función sí estaba probada de forma aislada (en `test_qe_schema_consistency.py`), que es precisamente por lo que la prueba pasaba mientras el punto de llamada estaba roto.

Corrección. Se importa la función correcta y el contenido se calcula antes de abrir el archivo, de modo que un fallo ya no puede truncar el artefacto.

Pruebas de regresión. La prueba fuerza el fallo de la síntesis y exige que la ejecución *complete*, que el esquema no esté vacío y que describa las entidades de *este* blueprint.

D-04 — El fallo más ruidoso escapaba del manejador y no lo veía nadie

Severidad: alta. Dejaba la sesión en `RUNNING` indefinidamente.

Evidencia. La prueba del modo `MODEL` con un conjunto de instrucciones ilegible esperaba una sesión marcada como bloqueada con la causa registrada. En su lugar, la excepción escapó de `_execute_pipeline_steps`.

Causa raíz. El `try` del pipeline empezaba *después* de la selección de modo y de la carga del conjunto de instrucciones. El código re-lanza deliberadamente en modo `MODEL` para fallar con fuerza (invariante 11) — y eso es correcto —, pero al estar fuera del manejador la excepción moría con el hilo demonio. La fila de sesión se quedaba en `RUNNING` y el operador miraba un pipeline que ya se había detenido. Lo mismo aplicaba a una clave de API ambigua, que `detect_provider` rechaza a propósito.

Corrección. El manejador ahora envuelve también la decisión de modo y la carga de instrucciones. Fallar con fuerza sigue siendo el comportamiento; fallar *en silencio para el operador* no.

Pruebas de regresión. Dos pruebas: el conjunto de instrucciones ilegible en modo `MODEL` y la clave ambigua. Ambas exigen estado `FAILED` y la causa en `error_message`.

D-02 — La razón del bloqueo se calculaba y se descartaba

Severidad: alta. Afecta al Principio V y a la experiencia del operador.

Evidencia. La prueba del límite de 3 auto-reparaciones afirmaba que el texto de la razón del bloqueo llegara al cliente. Falló: `state.phases[4].blocking_reason` era `None`. La fase se reportaba correctamente como `BLOCKED`, pero sin causa.

Causa raíz. Ambas rutas de bloqueo asignaban una variable local `blocked_reason`, mientras que el objeto `PhaseState` se construye a partir de `reason`. `blocked_reason` se escribía tres veces (inicialización y las dos rutas de bloqueo) y **nunca se leía**. El resultado en la API era `blockingReason: null` para la fase 5 detenida por el límite constitucional y para la fase 6 detenida por un Quality Gate bloqueado.

Corrección. Ambas rutas asignan ahora `reason`, que es el campo que se serializa. Se eliminó la variable muerta.

Pruebas de regresión. Las pruebas afirman el *contenido* del mensaje, no sólo el estado: “3 auto-reparaciones” y el mensaje del Quality Gate deben aparecer en `blocking_reason`.

D-01 — Funciones duplicadas: código muerto con contrato divergente

Severidad: media. Deuda técnica con riesgo de deriva silenciosa.

Evidencia. Al preparar las pruebas de fases obsoletas, la cobertura mostró líneas imposibles de alcanzar. La inspección del archivo reveló que `mark_downstream_outdated` y `clear_outdated_phases` estaban definidas **dos veces** cada una en el mismo módulo (líneas 325/351 y 437/478).

Causa raíz. Python enlaza el nombre a la *última* definición, de modo que el primer par era código muerto: 34 sentencias que ninguna prueba podía ejecutar. No era redundancia inocua: los dos contratos diferían. La versión sepultada `mark_downstream_outdated` devolvía `None` y *sobrescribía* el conjunto de fases obsoletas; la vigente devuelve la lista acumulada y la *extiende*. Un llamador que se guiara por la primera firma habría escrito `if mark_downstream_outdated(...) is None` y obtenido la respuesta contraria.

Corrección. Se eliminó el par muerto. El comportamiento observable no cambia (el nombre ya apuntaba a la segunda definición), y el recuento de sentencias del módulo bajó de 301 a 267.

Pruebas de regresión. Una prueba comprueba sobre el código fuente que cada función está definida una sola vez; otra fija el tipo de retorno (lista) de la función vigente.

7 Resultados de ejecución

Antes y después

Medición sobre el mismo entorno. “Antes” corresponde al commit `2bddc0a` (`fix: offline mode said it answered the prompt; a badge claimed verification`), el HEAD inmediatamente anterior a esta campaña, sin ninguno de sus cambios.

Métrica	Antes	Después	Variación
Pruebas de backend correctas	589	825	+236
Pruebas de backend omitidas	3	3	—
Pruebas de frontend correctas	64 (10 archivos)	73 (11 archivos)	+9
Cobertura global del backend	85,3 %	89,6 %	+4,3 pp
Sentencias sin cubrir	994	727	−267

Dos advertencias de lectura, para no atribuir a esta campaña más de lo que le corresponde:

- Otra sesión de trabajo entregó cambios en paralelo sobre el mismo árbol (por ejemplo en `orchestrator/stages/runner.py`), de modo que el movimiento de la cobertura *global* no es atribuible sólo a estas pruebas. Las cifras por módulo de la tabla siguiente sí lo son: cada una se midió antes y después sobre el mismo módulo.
- La cifra de “antes” de las pruebas de backend es el mismo punto de partida para las dos fases; el total combina ambas.

Cobertura por módulo priorizado

Módulo	Antes	Sin cubrir	Después	Sin cubrir
<code>services/docker_service.py</code>	52,7 %	69	100,0 %	0
<code>api/routes_session.py</code>	71,3 %	78	100,0 %	0
<code>services/lifecycle_service.py</code>	76,4 %	71	100,0 %	0
<code>services/pipeline_runner.py</code>	82,4 %	61	100,0 %	0

`cost/mlflow_sink.py` ya estaba al 100 % antes de la campaña (33 sentencias); la corrección de D-03 le añadió 9 sentencias y todas quedan cubiertas (42 de 42).

Frontend: qué se añadió y qué sigue sin cubrirse

El frontend pasó de 64 a 73 pruebas. Las nuevas no son más pruebas de componente: son *flujos* que ejecutan los módulos de servicio reales, el cliente HTTP real y las vistas reales contra un backend

simulado en la frontera HTTP (MSW). Esto es lo que faltaba: las 64 pruebas anteriores sustituyen la capa de servicios con `vi.mock`, así que ninguna construya una petición. Un backend que renombrara un parámetro habría mantenido las 64 en verde.

Lo que ahora queda fijado, y que antes no lo estaba: la URL, el método, los parámetros y el cuerpo de cada petición del flujo; que la credencial viaja en una cabecera y *no* se persiste en ningún almacenamiento (Principio VI); que un fallo del backend se propaga en lugar de inventarse; y — de extremo a extremo en la interfaz — que el smoke test informa “no ejecutado” cuando el endpoint no responde, en vez de un éxito que nadie observó. Ese último caso no tenía *ninguna* prueba pese a estar documentado como corregido en `docs/frontend_audit.md`.

Sigue sin cubrirse el navegador real: no hay Playwright ni Cypress. Estas pruebas ejercen el flujo con jsdom y un backend simulado, no con un navegador y un backend vivos.

Reproducción

```
# Suite de backend con cobertura
.venv/bin/python -m pytest -q \
  --cov=backend/app --cov-report=term-missing:skip-covered

# Suite de frontend
cd frontend && npm test
```

Evidencia versionada en `reports/qe/`:

- `baseline-run.txt` — salida completa antes (commit 2bddc0a), con la tabla de líneas sin cubrir por archivo.
- `after-run.txt` — salida completa después.
- `coverage-summary.json` — cobertura por archivo en ambos puntos, en formato consumible.
- `hang-trace.txt` — traza del cuelgue que motivó D-03.

8 Brechas restantes y recomendaciones

Tras esta campaña, los módulos con más sentencias sin cubrir son:

Sin cubrir	Cobertura	Sentencias	Módulo
67	70,9 %	230	<code>services/architecture_service.py</code>
62	63,1 %	168	<code>services/requirements_service.py</code>
53	83,3 %	317	<code>services/model_sql_service.py</code>
44	70,9 %	151	<code>orchestrator/stages/journal.py</code>
34	91,6 %	403	<code>orchestrator/stages/runner.py</code>
29	25,6 %	39	<code>ssl_compat.py</code>
26	90,3 %	268	<code>services/security_service.py</code>

Recomendación de prioridad para la próxima campaña:

1. `architecture_service.py` y `model_sql_service.py` — serialización de OpenAPI, Mermaid y DDL/JPA. Son artefactos entregables; un error de serialización se descubre tarde y en el cliente. Este es el mismo perfil que D-05: código que produce archivos que nadie comprueba hasta el final.
2. `requirements_service.py` — descomposición y refinamiento con LLM.
3. `journal.py` — bordes de presupuesto y serialización del diario de ejecución.

`ssl_compat.py` se recomienda **no** perseguir: es compatibilidad TLS en tiempo de importación y su cobertura tiene poco valor de producto.

El siguiente salto de valor no es más cobertura de línea, sino cerrar la brecha de la Sección 9: pruebas de aceptación HTTP independientes contra el microservicio generado y desplegado.

9 Limitaciones del informe

Cubierto no es correcto. La cobertura mide qué líneas se ejecutaron, no qué afirmaciones son verdaderas. Las pruebas fijan el comportamiento *actual* del sistema frente a los contratos declarados; no demuestran que esos contratos sean los deseados.

Hay una limitación específica y más profunda que conviene que el Project Manager conozca: **la verificación que la plataforma hace del código que ella misma genera es autorreferencial**. El sistema escribe las pruebas Mockito y después las ejecuta; un `mvn test -o` en verde demuestra que el código hace lo que dicen las pruebas de la plataforma, no lo que dicen los escenarios de aceptación del blueprint. Cerrar esa brecha es un entregable aparte: pruebas de aceptación HTTP independientes, una por escenario declarado, contra el servicio desplegado. Queda explícitamente fuera del alcance de esta campaña y se recomienda como siguiente paso de mayor valor.

En el frontend, las 73 pruebas ya no son sólo de componente: 9 de ellas recorren flujos completos contra un backend simulado en la frontera HTTP. Pero **no hay navegador real**: no se usa Playwright ni Cypress, y el backend es un simulador, no un servidor vivo. Un fallo que sólo aparezca en el navegador (CSS, foco, un evento nativo) no lo detectarían.